

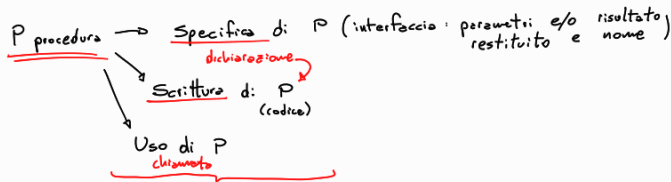
ASTRAZIONE (del codice)

- ↳ ignorare dettagli non rilevanti
- ↳ identificare proprietà di interesse
- variabili = astrazione della cella di memoria
- procedure = astrazione del codice

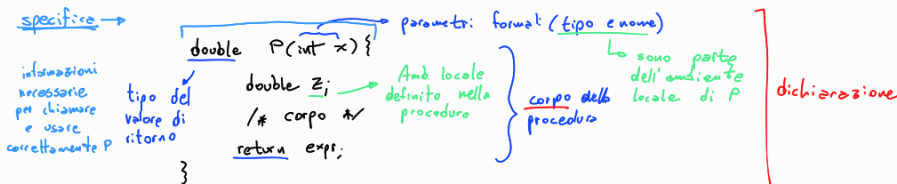
⇒ All'astrazione si attribuisce un NOME

- Parametri: permettono di rappresentare con unico codice una funzione dipendente da valori di input
unico codice = più computazioni possibili
- Corpo: insieme delle istruzioni che vengono eseguite quando si riferisce il nome della procedura

- Una procedura ha una singola entrata
- L'esecuzione del codice chiamante viene sospesa per passare il controllo alla procedura chiamata
- Al termine della procedura il controllo torna sempre al chiamante che riprende l'esecuzione (processo automatico)



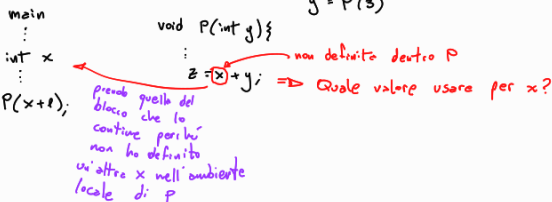
Abbiamo bisogno di strumenti sintattici per fare questi passaggi:



Uso di P → chiamata

Main ...
double y;
y = P(3);

parametri attuali



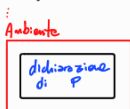
Regole di visibilità sono necessarie per risolvere i riferimenti non locali (non definiti nella procedura e non parametri formali)

⇒ rimangono le stesse viste: Ogni dichiarazione è visibile nel blocco in cui è definita e nei blocchi annidati che non la sovrastano

statico

Annidamento delle definizioni del blocco

⇒ Dove si trova la dichiarazione



regole di SCOPING statico

dinamico

Annidamento della chiamata e quindi dell'esecuzione del blocco

⇒ Dove si trova la chiamata



regole di SCOPING dinamico

⇒ Regole per l'ambiente di riferimento

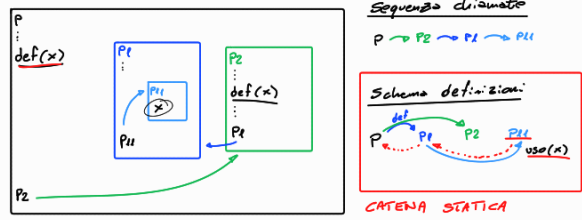
regole di scope + passaggio di parametri + Regole di binding

REGOLE DI SCOPE

```
{ int x = 0;
void foo () {
    x++;
}
void foo () {
    int x = 0;
    foo();
}
foo();
}
```

Annotations:
- `int x = 0;`: Ambiente che contiene la definizione (statico). L'ambiente non dipende dalla chiamata.
- `x++`: x non locale, dove trova significato?
- `int x = 0;` inside `foo()`: Ambiente che contiene la chiamata (dinamico). L'ambiente di riferimento cambia (potenzialmente) da ogni chiamata.

SCOPING STATICO



```
{ int x = 0;
void pippo (int n) {
    x = n + 1;
}
pippo(3);
write(x);
}

{ int x = 0;
  pippo(3);
  write(x);
}
```

Annotations:
- `x = n + 1;`: x non locale. => scoping statico ovvero x dentro pippo fa sempre riferimento alla x globale che ha valore 4.
- `write(x);`: refers to global x (value 4).
- `write(x);` in the second block: refers to local x (value 3).
- `write(x);` in the first block: refers to global x (value 0).

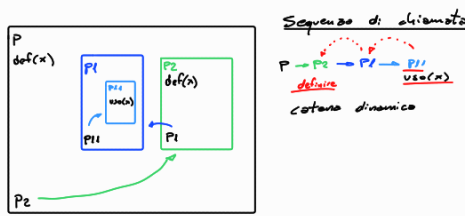
Nello scoping statico l'ambiente di riferimento e' indipendente dalla posizione della chiamata

NO:

```
int x;
int n;

void tr() {
    g();
}
void g() {
    tr();
}
```

SCOPING DINAMICO



```
{ int x = 0;
void pippo (int n) {
    x = n + 1;
}
pippo(3);
write(x);
}

{ int x = 0;
  pippo(3);
  write(x);
}
```

Annotations:
- `write(x);` in the first block: refers to global x (value 0).
- `write(x);` in the second block: refers to local x (value 3).
- `pippo(3);`: refers to global x (value 0).